

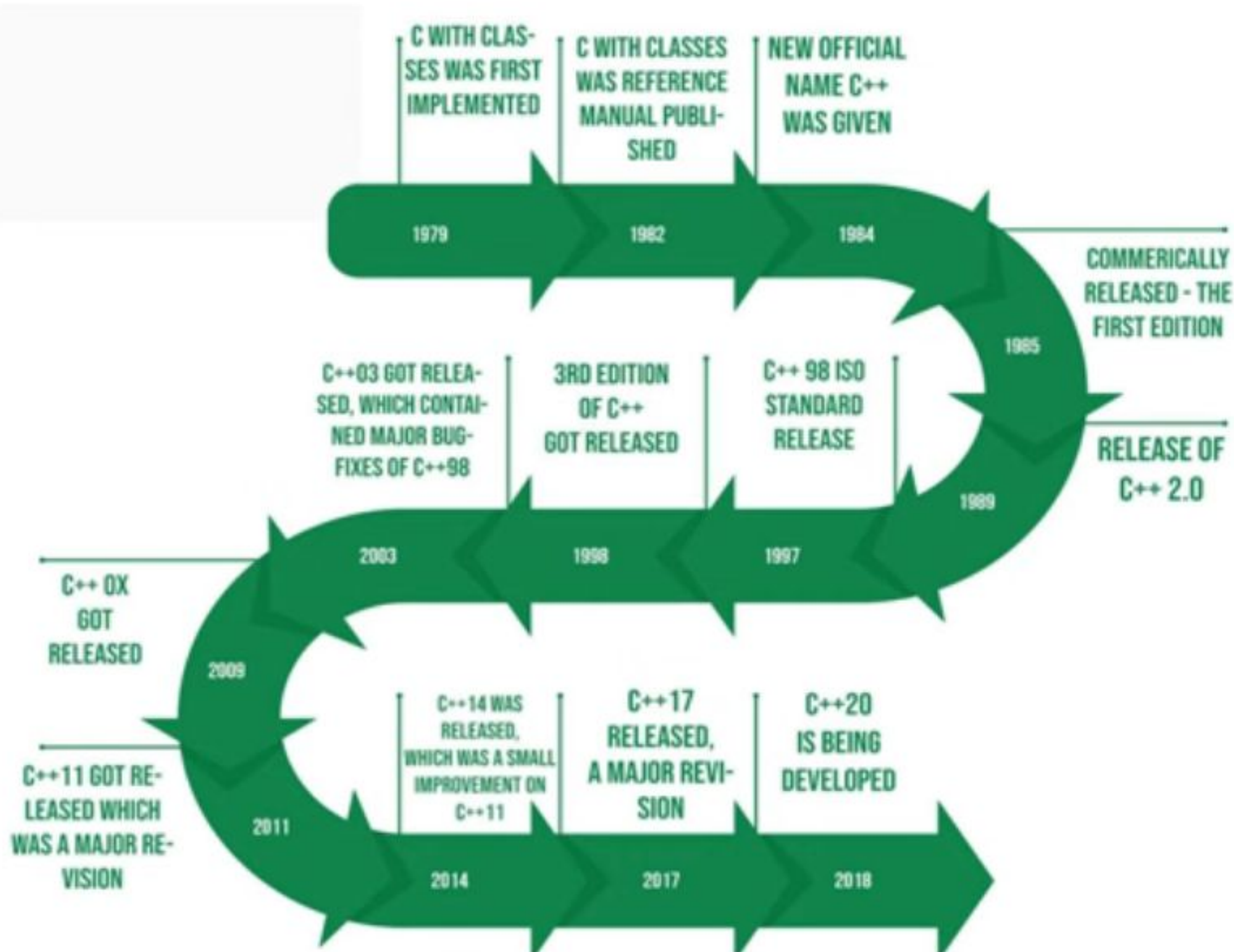
LINGUAGEM DE PROGRAMAÇÃO C++

Curso de Capacitação em Desenvolvimento Full Stack - MÓDULO 08 (40H)

Elaborado por: Prof(a). Esp. Érika D. G. R. Dos Santos

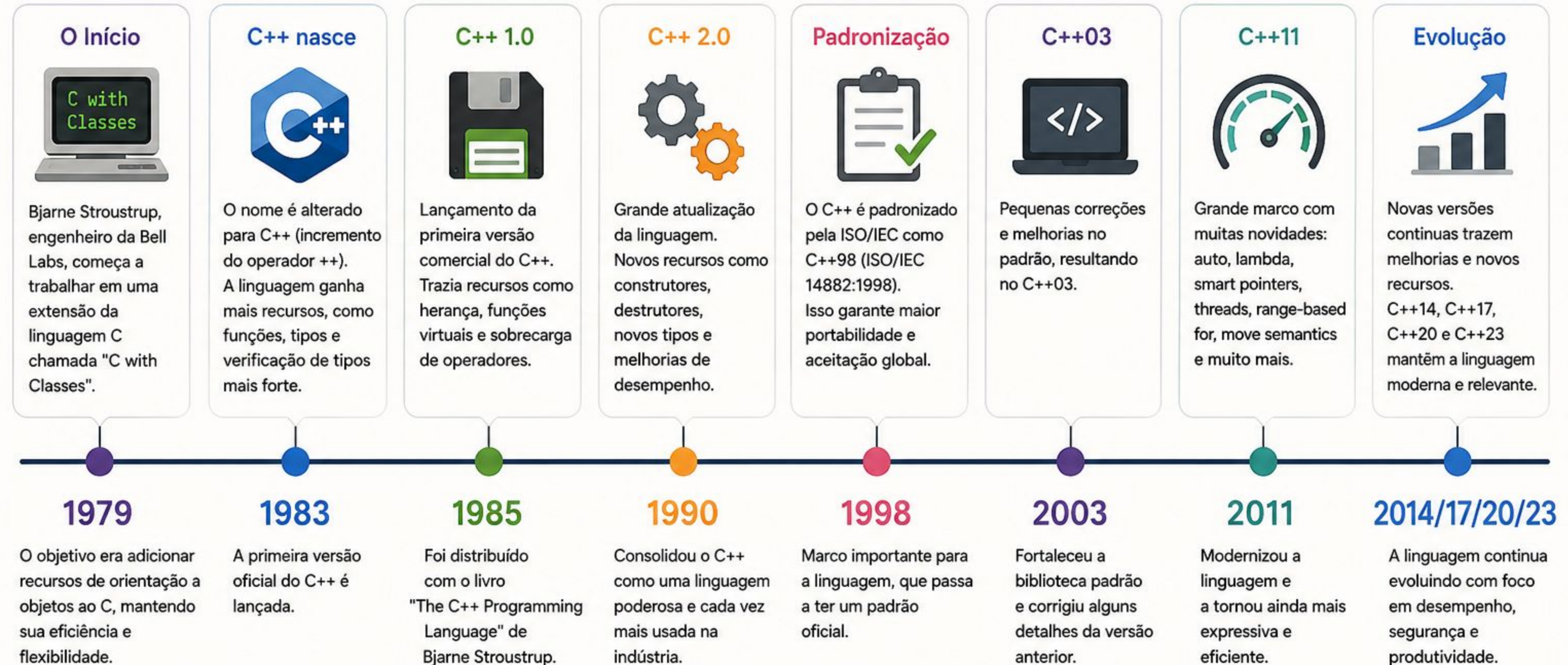
- Linguagem flexível, dinâmica, semelhante a C.
- Com verificação de tipo ativa.
- Herança.básica.
- Argumento de funcionamento padrão.
- Com classes e mais.
- O nome C++ veio mesmo apenas em 83 e quer dizer maior que C.
- É uma linguagem compilada multiparadigma, não é só usada com Orientação a Objetos e de uso geral.

History of C++



A História do C++

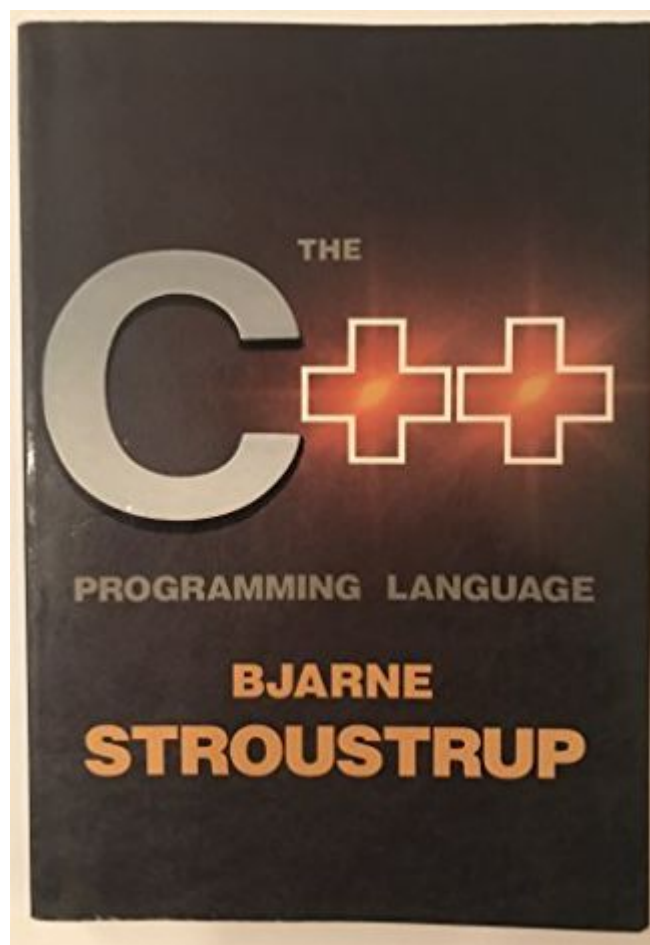
A jornada de uma das linguagens de programação mais influentes do mundo.



E até hoje...

C++ é amplamente utilizada em sistemas, jogos, aplicações de alta performance, embarcados, finanças e muito mais. Sua combinação de performance, controle e abstração continua única e poderosa.





1o livro criado pelo criador do linguagem e grande referencia.

C++ é fortemente **Tipada** e **Estática**.

Quando a linguagem é fortemente Tipada ela não permite a mundaça de um tipo para outro.

Nas linguagens dinâmicas o programa detecta na 1a atribuição de valor, já na estática, é sempre necessário informar o tipo na declaração da variável que será utilizada.

Ela é multiparadigma mas o forte dela esta na programação OO.

O C++ é utilizado na

Comunicação entre máquinas, sistemas embarcados, sw academicos, corporativos edição de fotos, vídeos, sistemas operacionais.



```
#include <stdio.h>
```

```
int main() {
```

```
    printf("Hello, World!");  
    return 0;
```

```
}
```



```
#include <iostream>
```

```
int main() {
```

```
    std::cout << "Hello World!";  
    return 0;
```

```
}
```

C++ DATATYPES



BUILT-IN / PRIMITIVE

DERIVADOS

ARRAYS
PONTEIROS
REFERÊNCIAS
FUNÇÕES

USER-DEFINED

STRUCTURES
UNIONS
CLASSES
ENUMERATIONS

INTEGER

FLOATING

VOID

INT

CHAR

FLOAT

DOUBLE





C/C++ DATA TYPES

QUICK REFERENCE TABLE

 TYPE	 TYPICAL SIZE (BYTES)	 TYPICAL RANGE
char	1byte	-128 to 127
signed char	1byte	-128 to 127
signed char	1byte	-128 to 127
int	4bytes	-2147483648 to 2147483647
unsigned int	4bytes	0 to 4294967295
signed int	4bytes	-2147483648 to 2147483647
short int	2bytes	-32768 to 32767
unsigned short int	2bytes	0 to 65,535
signed short int	2bytes	-32768 to 32767
long int	8 bytes	-9,223,372,036,854,775,808 to 9,223,372,036,854,775,807 (-2 ⁶³ to 2 ⁶³ -1)
signed long int	8 bytes	same as long int
unsigned long int	8 bytes	0 to 18,446,744,073,709,551,615 (0 to 2 ⁶⁴ -1)
long long int	8 bytes	-9,223,372,036,854,775,808 to 9,223,372,036,854,775,807 (-2 ⁶³ to 2 ⁶³ -1)
float	4bytes	~1.18e-38 to ~3.40e+38
double	8 bytes	~2.23e-308 to ~1.79e+308
long double	12 bytes	~3.37e-4932 to ~1.19e+4932
wchar_t	2 or 4 4 bytes	1 wide character (platform dependent)

*Typical sizes are for modern 64-bit platforms. Platform details may vary.

adopted when starting to build a new software system. The definition of the TIOBE index can be found [here](#).

Jun 2026	Jun 2025	Change	Programming Language		Ratings	Change
1	1			Python	18.96%	-6.91%
2	3	▲		C	10.77%	+1.30%
3	2	▼		C++	8.03%	-2.65%
4	4			Java	7.90%	-0.94%
5	5			C#	4.85%	+0.17%
6	6			JavaScript	3.04%	-0.17%
7	8	▲		Visual Basic	2.80%	+0.59%
9	12	▲▲		SQL	1.77%	+0.23%

1998 – C++98 e a STL

A padronização do C++ trouxe a **STL (Standard Template Library)** para a biblioteca padrão da linguagem.

Principais recursos:

- **Containers:** `vector`, `list`, `map`
- **Algoritmos:** `sort`, `find`, `count`
- **Iteradores:** acesso genérico às estruturas de dados
-

Impacto: aumentou a produtividade dos desenvolvedores e reduziu a necessidade de implementar estruturas de dados do zero.

PONTEIROS E ALOCAÇÃO DINÂMICA

AULA 8: LINGUAGEM DE PROGRAMAÇÃO C++

Ponteiros

Trabalhando com Endereços de Memória

- Ponteiro: variável que armazena um endereço de memória
- Declaração: `tipo* nome;` (ex: `int* ptr;`)
- Operador endereço: `&` (obtém o endereço)
- Operador dereferenciamento: `*` (acessa o valor no endereço)

```
int x = 10;
```

```
int* ptr = &x;
```

```
cout << *ptr; // exhibe 10
```


Alocação Dinâmica

new e delete

- Memória alocada em tempo de execução (heap)
- `new`: aloca memória
- `delete`: libera memória

```
int* vetor = new int[10]; // aloca vetor de 10 inteiros
```

```
// ... uso do vetor
```

```
delete[] vetor; // libera memória
```

- Cuidado com vazamento de memória (memory leak)!

Testes de Mesa com Ponteiros

Rastreando Endereços e Valores

- Anotar endereços e valores a cada instrução
- Exemplo prático: trocar valores usando ponteiros

<https://www.youtube.com/watch?v=qs6qW4aDIaE>

Exercício 1: Troca de valores

Sem usar variáveis auxiliares (apenas ponteiros), implemente uma função que troca os valores de duas variáveis inteiras.

```
void trocar(int* a, int* b) {
```

```
    // Implemente aqui (sem criar novas variáveis!)
```

```
}
```

Exercício 2: Ponteiro para ponteiro (modificando o ponteiro original)

Implemente uma função que "estoura" o valor de um ponteiro, fazendo ele apontar para um novo inteiro alocado dinamicamente, sem retornar nada.

```
void criarNovoInteiro(int** ptr, int valor) {  
    // Aloque novo inteiro dinamicamente  
    // Faça ptr apontar para ele  
    // Atribua 'valor' a esse novo inteiro  
}
```

Exercício 3: Ponteiro como "opcional" (nullable)

Enunciado:

Implemente uma calculadora segura que recebe um ponteiro para o resultado. Se o ponteiro for `nullptr`, a função apenas retorna se a operação foi possível; caso contrário, armazena o resultado.

// Retorna true se a operação foi bem-sucedida (divisão por zero = false)

bool calcular(int a, int b, char operador, int* resultado) {

// Se resultado != nullptr, armazene o valor lá

// Retorne false se operação inválida (divisão por zero, operador desconhecido)

Vetores Estáticos (Tamanho Fixo)

```
#include <iostream>
```

```
using namespace std;
```

```
int main() {
```

```
    int numeros[5] = {10, 20, 30, 40, 50};
```

```
    cout << "Primeiro elemento: " << numeros[0] << endl;
```

```
    cout << "Último elemento: " << numeros[4] << endl;
```

```
    cout << "Todos os elementos: ";
```

```
    for(int i = 0; i < 5; i++) {
```

```
        cout << numeros[i] << " ";
```

```
    }
```

```
    cout << endl;
```

```
    numeros[2] = 99;
```

```
    cout << "Após modificar: " << numeros[2] << endl;
```

```
    return 0;
```

```
}
```

Vetores Dinâmicos

```
#include <iostream>
```

```
#include <vector>
```

```
using namespace std;
```

```
int main() {
```

```
    vector<int> valores;
```

```
    valores.push_back(10);
```

```
    valores.push_back(20);
```

```
    valores.push_back(30);
```

```
    cout << "Primeiro: " << valores[0] << endl;
```

```
    cout << "Tamanho: " << valores.size() << endl;
```

```
    cout << "Elementos: ";
```

```
    for(int i = 0; i < valores.size(); i++) {
```

```
        cout << valores[i] << " ";
```

```
    }
```

```
    cout << endl;
```

```
    valores.pop_back();
```

```
    cout << "Após remover: ";
```

```
    for(int x : valores) {
```

```
        cout << x << " ";
```

```
    }
```

```
    cout << endl;
```

```
    return 0;
```

```
}
```